Utveckling av Tetris i JavaFx

Från logik till grafiskt gränssnitt

Oliver Kalthoff
2026-04-17

Examensarbete vid Grit Academy
Klass: Java24
Examinator och handledare: Martin Haagen

Sammanfattning

Detta examensarbete beskriver utvecklingen av en applikation i form av spelet Tetris, byggt med Java och JavaFX. Projektet valdes för att utforska tekniker utanför de ordinarie ramarna för utbildningen till fullstack javautvecklare, med ett särskilt fokus på spel-loopar, realtidshantering och grafiska användargränssnitt. Genom att tillämpa objektorienterade principer har en tydlig separation mellan spellogik och rendering skapats. Resultatet är en fullt fungerande applikation som inkluderar mekaniker för rotation, kollisionsdetektering och poängräkning. Arbetet visar hur tekniker som AnimationTimer och enhetstester kan skapas för att skapa en stabil spelupplevelse.

Abstract

This thesis details the development of a Tetris application using Java and JavaFX. The project was undertaken to explore software development areas beyond the standard fullstack Java curriculum, focusing specifically on game loops, real-time state management and graphical user interfaces. By applying object-oriented patterns, a clear separation between game logic and the UI layer was achieved. The resulting application features core mechanics such as block rotation, collision detection and a progressive scoring system. The project demonstrates the effective use of JavaFX components such as AnimationTimer and the importance of unit testing in ensuring application stability in a real-time environment.

Förord

Detta examensarbete markerar slutet på min utbildning till Fullstack Javautvecklare vid Grit Academy. Arbetet har varit en spännande möjlighet att kliva utanför de tekniker vi främst fokuserat på under utbildningen och istället utforska spelutveckling och grafiska gränssnitt, som parallellt med programmering har varit något jag länge varit intresserad av att prova på.

Jag vill rikta ett stort tack till min handledare och examinator, Martin Haagen, för vägledning och stöd under utbildningens gång. Jag vill även tacka mina studiekamrater för givande diskussioner och feedback i klassrummet under utbildningstiden.

Slutligen vill jag tacka Grit Academy för två lärorika år som har gett mig en stabil grund att stå på inför min framtida yrkesroll som systemutvecklare.

Malmö, April 2026
Oliver Kalthoff

Innehållsförteckning

Sammanfattning 2
Abstract 2
Förord 3
Innehållsförteckning 4
Inledning 5
Teori 7
Metod 9
Resultat 11
Slutsats och diskussion 13
Referenser 15

1. Inledning

I detta kapitel presenteras bakgrunden till examensarbetet, dess syfte och de frågeställningarna som arbetet ämnar besvara. Vidare redogörs för de avgränsningar som gjorts för att hålla projektet inom givna tidsramar.

1.1 Bakgrund

Inom ramen för utbildningen till Fullstack Javautvecklare ligger tyngdpunkten främst på webbaserade applikationer, databashantering och serverlogik. För detta examensarbete valdes dock att utforska ett område som ligger utanför de ordinarie ramarna: spelutveckling med ett grafiskt användargränssnitt. Valet föll på att utveckla en version av det klassiska spelet Tetris, då det ställer höga krav på logikhantering, realtidsuppdateringar och en tydlig separation mellan applikationens tillstånd och dess visuella presentation.

1.2 Syfte

Syftet med projektet är att fördjupa kunskaperna inom objektorienterad programmering genom att lösa de unika utmaningar som uppstår vid utveckling av ett rutnätsbaserat spel. Målet är att skapa en stabil applikation med JavaFX som hanterar komplex logik för rörelse, rotation och kollisionsdetektering, samtidigt som principer för ren kod och en tydlig systemarkitektur efterlevs.

1.3 Frågeställning

För att styra arbetet framåt har följande frågeställningar definierats:

- Hur kan en effektiv spelloop implementeras i JavaFX för att hantera realtidslogik oberoende av bilduppdateringsfrekvens?

- Hur bör systemarkitekturen struktureras för att uppnå en tydlig separation mellan spellogik (model) och användargränssnitt (view)?

- Vilka tekniska utmaningar uppstår vid implementering av kollisionsdetektering och radrensning i ett 2D-rutnät, och hur kan dessa lösas?

1.4 Avgränsningar

För att säkerställa att projektet kan genomföras under den avsatta tiden har följande avgränsningar gjorts:

- Applikationen saknar ljudhantering och musik.
- Ingen funktionalitet för multiplayer eller lagring av poäng i en extern databas har implementerats
- Grafiken är begränsad till enkla geometriska former utan avancerade texturer eller partikeleffekter.
- Spelet följer grundläggande regler för Tetris men implementerar inte avancerade rotationssystem som SRS (Super Rotation System)

2. Teori

Detta kapitel redogör för de tekniska ramverk och mjukvaruarkitektoniska principer som tillämpats under utvecklingen. Fokus ligger på JavaFX som grafikmotor samt designmönster för att separera logik från presentation.

2.1 JavaFX och GUI-utveckling

JavaFX är ett ramverk för att bygga klientapplikationer i Java. Till skillnad från dess föregångare Swing, erbjuder JavaFX en mer flexibel hantering av grafik genom användning av en ett verktyg kallat Scene Graph. För att definiera användargränssnittet används ofta FXML, ett XML-baserat språk som möjliggör en tydlig separation mellan design och programkod. I en spelapplikation ger JavaFX tillgång till komponenter som Canvas, vilket tillåter en att rita på pixelnivå, vilket är nödvändigt för att rendera rörliga objekt med hög precision.

2.2 Model-View-Controller (MVC)

För att skapa en välstrukturerad och underhållsvänlig applikation tillämpas designmönstret Model-View-Controller. Mönstret delar upp applikationen i tre distinkta lager:

- **Model:** Innehåller applikationens data och affärslogik (t.ex. regler för Tetris och spelplanens tillstånd).
- **View:** Ansvarar för den visuella representationen av datan (gränssnittet som användaren ser).
- **Controller:** Fungerar som en länk mellan Model och View. Den tar emot indata från användaren och uppdaterar modellen, som i sin tur reflekteras i vyn.

Denna separation underlättar enhetstester och gör det möjligt att ändra det visuella gränssnittet utan att påverka de bakomliggande reglerna.

2.3 Spelloopar och AnimationTimer

En central del i spelutvecklingen är spelloopen. Det är en kontinuerlig cykel som hanterar indata, uppdaterar spelets tillstånd och renderar grafiken på skärmen. I JavaFX implementeras detta effektivt med klassen AnimationTimer. Denna klass är synkroniserad med skärmens uppdateringsfrekvens (vanligtvis 60 bilder per sekund), vilket säkerställer mjuka animationer. Genom att mäta tidsskillnaden (delta time) mellan varje uppdatering kan spelets logik, såsom blockens fallhastighet, styras oberoende av datorns prestanda.

2.4 Rutnätsbaserad logik och kollisionsdetektering

Tetris bygger tekniskt på ett 2D-rutnät (grid). Varje position i rutnätet kan antingen vara tom eller innehålla ett block. Kollisionsdetektering i denna miljö sker genom att matematiskt kontrollera om en spelpjäs (Tetromino) koordinater överlappar med redan fyllda celler i rutnätet eller spelets yttre gränser. Genom att representera pjäserna som matriser kan rotationer och förflyttningar beräknas innan de faktiskt genomförs, vilket möjliggör en stabil logik för att förhindra att objekt passerar genom varandra.

3. Metod

Detta kapitel beskriver den praktiska genomförandet av projektet, från planering och val av verktyg till den faktiska implementeringen och kvalitetssäkringen av mjukvaran.

3.1 Arbetsprocess

Projektet genomfördes under en period av 90 timmar och strukturerades enligt en agil arbetsprocess med korta sprintar. Arbetet inleddes med en planeringsfas där projektets omfattning och avgränsningar definierades utifrån de krav som ställts i det initiala ämnesvalet. En tidsbudget om 90 timmar upprättades, där utrymme medvetet lämnades för att hantera oförutsedda tekniska utmaningar och förbättringar.

Under projektets gång reviderades arbetsprocessen för att lägga ett större fokus på systemarkitektur och testbarhet. Denna justering gjordes för att säkerställa hög kodkvalitet och för att möjliggöra en djupare teknisk analys av applikationens stabilitet. Genom att utnyttja den planerade tidsbufferten kunde fokus skiftas mot att bygga en robust och vältestad kodbas, snarare än ett fokus på snabbt resultat.

Utvecklingsfasen skedde därefter inkrementellt, där funktionalitet lades till stegvis: från grundläggande spelplanslogik till avancerade funktioner som matrisbaserad rotation, kollisionsdetektering och ett dynamiskt poängsystem. En loggbok fördes under processen för att dokumentera tekniska vägval och beslut.

3.1.1 Tidsåtgång och resursfördelning

I enlighet med projektets instruktioner har arbetet loggförts för att skapa ett underlag som simulerar en professionell debiteringssituation. Den totala utvecklingstiden om 90 timmar har dedikerats till den tekniska framställningen och fördelats enligt följande:

- **Analys och arkitektur (15h)**: Kravställning, design av MVC-mönster och teknisk setup.
- **Logik och backend (35h)**: Implementering av GameBoard, matrisrotation och kollisioner.
- **GUI och integration (25h)**: FXML-design i Scene Builder och koppling till controllers.
- **Kvalitetssäkring (15h)**: Enhetstestning med JUnit, buggfixar och prestandaoptimering.

3.2 Teknikval och utvecklingsmiljö

Applikationen är utvecklad i Java 21 med JavaFX 17 som grafiskt ramverk. För projektstyrning och hantering av beroenden användes byggverktyget Maven. Valet av teknik baserades på utbildningens grundpelare i Java, men utökades med JavaFX för att möta projektets krav på ett grafiskt användargränssnitt.

3.3 Implementering

Implementeringen inleddes med att bygga upp spelets domänmodell, det vill säga de klasser som hanterar logik utan att känna till det grafiska gränssnittet. Centralt i detta arbete var klassen GameBoard, som representerar spelets 2D-grid som en tvådimensionell array av heltal. Genom att använda matriser för att definiera de olika tetromino-blocken kunde rotationslogik implementeras med hjälp av matris-transformationer, vilket minimerade behovet av hårdkodade positioner för varje rotationsläge.

För att hantera spelets realtidsaspekt används AnimationTimer. Utmaningen låg i att synkronisera bitarnas fallhastighet med användarens input, vilket löstes genom att loopen beräknar tidsdifferensen mellan varje anrop. Pjäsen flyttas endast när ett tröskelvärde nåtts, baserat på aktuell nivå. Rotationslogiken implementerades som en självrättande process där pjäsen transformeras och därefter omedelbart valideras. Om en kollision detekteras återställer systemet pjäsen till dess senaste giltiga position.

Gränssnittet konstruerades parallellt i Scene Builder. FXML användes för att definiera strukturen, medan CSS användes för att separera den visuella stilen från logiken. Genom att använda Canvas-komponenten i JavaFX kunde renderingen optimeras genom att endast rita om de delar av spelplanen som faktiskt förändrats under föregående spelsekvens.

4. Resultat

I detta kapitel presenteras den färdiga applikationen och resultatet av de tekniska lösningar som implementerats. Resultatet visas genom beskrivningar av funktionalitet, det grafiska gränssnittet samt utfallet av kvalitetssäkringen.

4.1 Den färdiga applikationen

Slutprodukten är en stabil och fullt fungerande Tetris-applikation byggd i JavaFX. Applikationen uppfyller samtliga mål som sattes upp i ämnesvalet, inklusive mekanik för fallande block, användarstyrd förflyttning, rotation och radrensning. Systemet är uppbyggt enligt en modulär arkitektur där logiken är helt separerad från det grafiska lagret, vilket möjliggör framtida expansioner eller byte av grafikmotor utan att kärnlogiken påverkas.

4.2 Spelmekanik och logik

Genomförandet har resulterat i följande kärnfunktioner:

- **Rörelse och kollision**: Ett robust system som validerar varje förflyttning mot spelplanens nuvarande tillstånd (grid) innan positionen uppdateras.
- **Rotation**: En matrisbaserad rotationslogik som hanterar samtliga sju tetromino-typer.
- **Progression**: Ett poängsystem som automatiskt ökar spelets svårihetsgrad var tionde rensad rad, vilket skapar en utmanande spelupplevelse.
- **Game Over**: Logik som känner av när nya klossar inte kan skapas och presenterar ett tydligt avslut för användaren med möjlighet till omstart.

4.3 Grafiskt användargränssnitt (GUI)

Gränssnittet har designats med fokus på tydlighet och användarvänlighet. Genom JavaFX har en vy skapats som innehåller:

- **Spelytan**: En central canvas där spelet renderas i realtid.
- **Statistikpanel**: Tydlig visning av nuvarande poäng, nivå och antal rensade rader.
- **Menysystem**: En startskärm och en avslutningsskärm som ramar in spelupplevelsen.

4.4 Testresultat

För att säkerställa systemets stabilitet och korrekthet har omfattande tester utvecklats med JUnit. Totalt har tre huvudområden testats: GameBoard, GameStats och TetrisGame. Resultaten visar att samtliga kritiska affärsregler efterlevs.

- **Logik för spelplanen (GameBoard)**: Tester verifierar att celler kan sättas och hämtas korrekt, samt att metoden för att rensa rader faktiskt identifierar fulla rader och flyttar ner ovanliggande block. Gränskontroller har bekräftat att systemet inte tillåter placering av block utanför spelplanens koordinater.

- **Progression och poäng (GameStats)**: Enhetstester har bekräftat att poängberäkningen fungerar korrekt och att svårighetsgraden ökar vid exakt var tionde rensad rad. Testerna har även verifierat att spelets fallhastighet minskar logiskt vid nivåökning men aldrig understiger det definierade minimivärdet, vilket garanterar spelbarhet även på höga nivåer.
- **Spelmekanik (TetrisGame)**: Genom simulerade spelmoment har testerna bekräftat att blockens rörelser och rotationer blockeras korrekt vid kollision med existerande block eller väggar. Hard drop funktionen har även bekräftats omedelbart generera en ny kloss efter att den föregående låsts fast.

Genom att använda @RepeatedTest för rotationslogiken har det säkerställts att slumpmässigt genererade pjäser behåller giltiga koordinater även efter flera sekvenser av rotationer. Att samtliga tester passerar deterministiskt ger en hög tillförlitlighet till applikationens robusthet.

5. Slutsats och diskussion

I detta avslutande kapitel analyseras resultatet i förhållande till de inledande frågeställningarna. Vidare förs en diskussion kring de valda metoderna, systemets arkitektur samt förslag på framtida utveckling.

5.1 Analys av frågeställningar

Arbetet har utgått från tre frågeställningar som här besvaras utifrån det färdiga resultatet:

- **Hur kan en effektiv spelloop implementeras i JavaFX?**
  Genom användning av AnimationTimer skapades en loop som är synkroniserad med skärmens uppdateringsfrekvens. Genom att mäta tidsskillnaden i nanosekunder i varje tick, kunde spelets logik göras helt oberoende av bildfrekvensen. Detta resulterade i en stabil spelupplevelse där hastigheten ökar linjärt med spelarens nivå utan att påverka animationernas responsivitet.

- **Hur bör systemarkitekturen struktureras för separation mellan logik och gränssnitt?**
  Genom att strikt tillämpa MVC-mönstret har spellogiken i TetrisGame och GameBoard hållits helt fri från referenser till JavaFX-komponenter. Detta har inte bara gjort koden mer överskådlig utan har varit en förutsättning för projektets höga testbarhet. Gränssnittet i FXML och deras tillhörande controllers fungerar endast som ett lager för presentation och användarinput.

- **Vilka tekniska utmaningar uppstår vid kollisionsdetektering och radrensning?**
  Den största utmaningen låg i att hantera rotationer nära spelplanens gränser. Genom att använda matriser för pjäser och en logik som korrigerar ogiltiga positioner, kunde kollisioner effektivt förhindras. Radrensningen löstes genom att iterera genom rutnätet efter varje fastlåst pjäs, vilket visade sig vara en prestandamässigt effektiv metod för ett 2D-spel av denna skala.

5.2 Arkitektur och testbarhet

En central del av projektet har varit att säkerställa hög kodkvalitet genom enhetstester. Efter feedback gällande arkitektur gjorde jag ett medvetet val att hålla klasserna GameBoard och GameStats som rena Java-klasser. Detta möjliggjorde omfattande tester med JUnit utan att behöva hantera JavaFX inblandning i logiken, även om motorn i TetrisGame använder AnimationTimer för att driva själva spelloopen.

Användningen av @RepeatedTest för rotationslogiken var väldigt värdefull. Det visade att systemet är robust även under stress och att de matematiska beräkningarna för matrisrotationen håller i alla scenarier. Att separera poänglogiken till en egen klass gjorde det också enkelt att verifiera spelets progression oberoende av själva spelandet.

5.3 Metoddiskussion och reflektion

Tidsbudgeten på 90 timmar visade sig vara väl avvägd för projektets omfattning. Vid projektets start gjordes ett medvetet val att inkludera marginaler i tidsplaneringen för att hantera den tekniska inlärningskurvan. Detta gav utrymme för att gå på djupet med enhetstester och att implementera lite extra funktioner som paus och startmenyer. Jag tycker att detta bekräftar vikten av en realistisk tidsplanering och självkännedom som prioriterar kvalitet och kodstabilitet framför enbart kvantitet av funktioner.

Att kliva utanför den ordinarie teknikstacken (fullstack Java) till JavaFX har varit en lärorik del av arbetet. Den största utmaningen har varit omställningen att hantera states i realtid, där varje bilduppdatering i spelloopen kräver omdelebar respons, jämfört med den mer statiska livscykeln i en traditionell webbapplikation.

Under utvecklingen har jag använt LLM:er (AI) som ett tekniskt bollplank, främst för att snabbt tyda dokumentation kring specifika JavaFX-komponenter som varit främmande och för felsökning. Det har dock varit tydligt att den arkitektoniska designen och separationen av ansvarsområden krävt manuella beslut för att säkerställa att systemet förblev testbart och modulärt enligt mina projektmål.

Referenser

Dokumentation och tekniska guider

- **OpenJFX.** *JavaFX Documentation*. https://openjfx.io/javadoc/17/ (2026-04)
- **OpenJFX.** *JavaFX CSS Reference Guide.* https://openjfx.io/javadoc/17/javafx.graphics/javafx/scene/doc-files/cssref.html (2026-04)
- **OpenJFX.** *JavaFX Introduction to FXML.* https://openjfx.io/javadoc/22/javafx.fxml/javafx/fxml/doc-files/introduction_to_fxml.html (2026-04)
- **Oracle.** *JavaFX Scene Builder User Guide.* *https://docs.oracle.com/javase/8/scene-builder-2/get-started-tutorial/jfxsb-get_started.htm* (2026-04)
- **JUnit Team.** *JUnit 5 User Guide.* https://junit.org/junit5/docs/current/user-guide/ (2026-04)

Videobaserat material

- **Baimagambetov, A (2019).** *Programming Tetris Game, Java (fx) Tutorial* Youtube. *https://www.youtube.com/watch?v=boAJUSN8fOU* (2026-04)

Programvara och verktyg

- **IntelliJ IDEA.** Jetbrains. (Utvecklingsmiljö).

- **Maven.** Apache Software Foundation. (Byggverktyg och beroendehantering).

- **Scene Builder.** Gluon. (Grafiskt layoutverktyg för FXML).